



GPT-Style LLM From Scratch

Complete Developer & Learner Documentation

Built by Arjun

Second-Year B.E. Information Technology Student

Every component implemented from first principles using PyTorch —

no black-box APIs, no shortcuts.

PyTorch

Python

Transformer

GPT-2

SFT

Ollama

This document serves as a conceptual guide to accompany the source code repository. It explains the theory, design decisions, and pipeline flow behind each component so learners can understand the project deeply without getting lost in implementation details.

■ Table of Contents

1 .	Project Overview & Motivation	pg. 3
2 .	Prerequisites & Background Knowledge	pg. 3
3 .	Tokenization & Preprocessing	pg. 4
4 .	Transformer Architecture	pg. 5
5 .	GPT-Style Model Design	pg. 7
6 .	Pretraining	pg. 8
7 .	Text Generation Strategies	pg. 9
8 .	GPT-2 Weight Loading	pg. 10
9 .	Checkpointing	pg. 11
1 0 .	SMS Spam Classification Fine-Tuning	pg. 11
1 1 .	Instruction Fine-Tuning (SFT)	pg. 13
1 2 .	Training & Evaluation Pipeline	pg. 15
1 3 .	Automated Response Evaluation with Ollama	pg. 16
1 4 .	End-to-End Pipeline Summary	pg. 17
1 5 .	Key Concepts Glossary	pg. 18
1 6 .	About the Author	pg. 19

1. Project Overview & Motivation

This project implements a complete GPT-style Large Language Model (LLM) entirely from scratch using PyTorch. The goal is to demystify how modern language models work by building every component — tokenization, attention mechanisms, training loops, fine-tuning, and evaluation — from first principles, without relying on high-level libraries like HuggingFace Transformers.

Most learners interact with LLMs through APIs or pre-built pipelines, which hides the mechanics. This project strips all of that away. By following it, you will understand exactly what happens when a language model reads text, makes predictions, learns from data, and follows instructions.

Why Build From Scratch?

- Understanding attention mechanisms at the tensor level, not just conceptually.
- Seeing how GPT-2 weights map to a custom architecture validates your implementation.
- Fine-tuning from a known checkpoint teaches transfer learning in practice.
- Instruction tuning and evaluation closes the loop on the full modern LLM pipeline.

2. Prerequisites & Background Knowledge

To get the most out of this project, you should be comfortable with the following:

Topic	Why You Need It
Python (intermediate)	All code is written in Python
PyTorch basics	Tensors, autograd, nn.Module, optimizers
Linear algebra	Matrix multiplication underlies all attention math
Probability & softmax	Token probability distributions, sampling
Basic neural networks	Forward pass, backpropagation, loss functions
Text tokenization concepts	How text becomes numbers

Recommended background reading: 'Attention Is All You Need' (Vaswani et al., 2017) and the original GPT papers by OpenAI. You do not need to understand every math detail before starting — the project itself builds intuition progressively.

3. Tokenization & Preprocessing

Before a neural network can process text, the text must be converted into numbers. Tokenization is the process of splitting raw text into discrete units (tokens) and mapping them to integer IDs that the model can work with.

3.1 What is a Token?

A token is the smallest unit of text the model operates on. Depending on the tokenizer, a token might be a full word, a subword fragment, a character, or even a byte. This project implements a word-level tokenizer using regular expressions as a learning exercise, and later uses OpenAI's tiktoken (BPE-based) for the GPT-2 compatible pipeline.

3.2 Vocabulary Construction

A vocabulary is a fixed mapping from every unique token to an integer index. The model only knows tokens in this vocabulary. A special (unknown) token handles words seen at inference time that were absent during training. Vocabulary size is a critical hyperparameter — larger vocabularies reduce unknown tokens but increase embedding table size.

3.3 Encoding & Decoding

Encoding converts raw text to a list of integer token IDs. Decoding reverses this — mapping integer IDs back to readable text. Both directions are needed: encoding for feeding data into the model, decoding for reading the model's generated output.

3.4 Sliding Window Dataset

For next-token prediction pretraining, the corpus is split into overlapping windows of fixed context length. Each window produces one training sample: the input is tokens 0..N-1 and the target is tokens 1..N (shifted by one). This teaches the model to predict the next token given all previous tokens in the context.

3.5 Embeddings

Token IDs are integers — they have no meaningful geometry. Embeddings convert each token ID into a dense vector of real numbers. The model learns these vectors during training. Additionally, positional embeddings encode where in the sequence each token sits, since the Transformer itself has no built-in notion of order.

Token Embedding: maps token ID → dense vector (shape: vocab_size × embed_dim)

Positional Embedding: maps position index → dense vector (shape: context_length × embed_dim)

Both are summed before entering the Transformer layers.

4. Transformer Architecture

The Transformer is the core neural architecture behind GPT. It was introduced in the paper 'Attention Is All You Need' (2017). Unlike RNNs, which process tokens sequentially, Transformers process all tokens in parallel using attention mechanisms.

4.1 Self-Attention

Self-attention allows each token to look at all other tokens in the sequence and decide how much 'attention' to pay to each one when computing its own representation. Every token produces three vectors from its embedding: a Query (what am I looking for?), a Key (what do I contain?), and a Value (what information do I pass on?).

Attention scores are computed by taking the dot product of each token's Query with every other token's Key, scaling by the square root of the head dimension to stabilize gradients, then applying softmax to get a probability distribution. The output is a weighted sum of all Value vectors according to these probabilities.

4.2 Causal (Masked) Attention

In a language model, when predicting the next token, the model must not be allowed to look at future tokens — that would be cheating. Causal attention applies a mask that sets attention scores to negative infinity for all future positions before the softmax step. After softmax, those positions get zero weight, effectively hiding them. This is implemented using a lower-triangular matrix of ones (all True positions can attend; upper triangle is masked).

4.3 Multi-Head Attention

Instead of running a single attention computation, Multi-Head Attention runs several attention operations in parallel — each called a 'head'. Each head learns to attend to different aspects of the input (e.g., one head might focus on syntactic relationships, another on semantic similarity). The outputs of all heads are concatenated and projected back to the model dimension. This gives the model richer representational power than a single attention mechanism.

4.4 Feed-Forward Network (FFN)

After attention, each token's representation passes through a small fully-connected feed-forward network applied independently to each position. This consists of two linear layers with a GELU activation in between. The FFN's hidden dimension is typically 4× the model's embedding dimension. This component adds non-linearity and allows the model to apply position-wise transformations.

4.5 GELU Activation

GELU (Gaussian Error Linear Unit) is the activation function used inside the FFN. Unlike ReLU which hard-clips negative values to zero, GELU smoothly gates values based on a Gaussian CDF. This smooth gating improves gradient flow during training and is the standard choice in GPT-style models.

4.6 Layer Normalization

Layer Normalization stabilizes training by normalizing activations across the feature dimension of each token independently. Unlike Batch Normalization (which normalizes across the batch), LayerNorm works on a per-token basis, making it well-suited for variable-length sequences. GPT uses a 'Pre-LN'

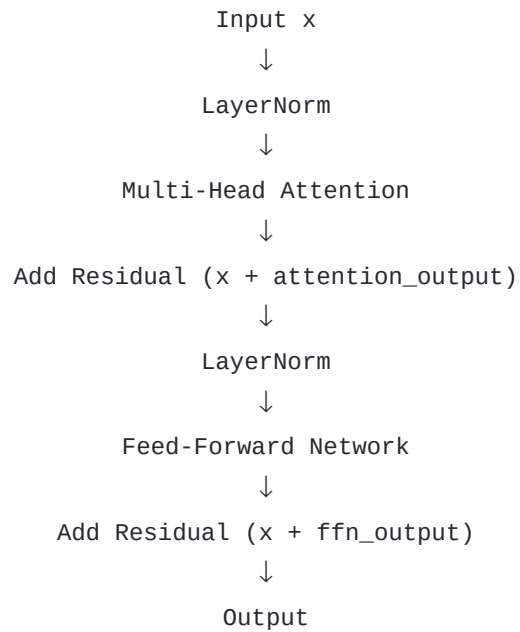
arrangement: LayerNorm is applied before each sub-layer (attention, FFN) rather than after.

4.7 Residual Connections

Each sub-layer (attention and FFN) is wrapped in a residual (skip) connection: the input to the sub-layer is added directly to its output. This allows gradients to flow directly through the network without passing through every transformation, enabling training of much deeper networks without vanishing gradients.

4.8 Full Transformer Block

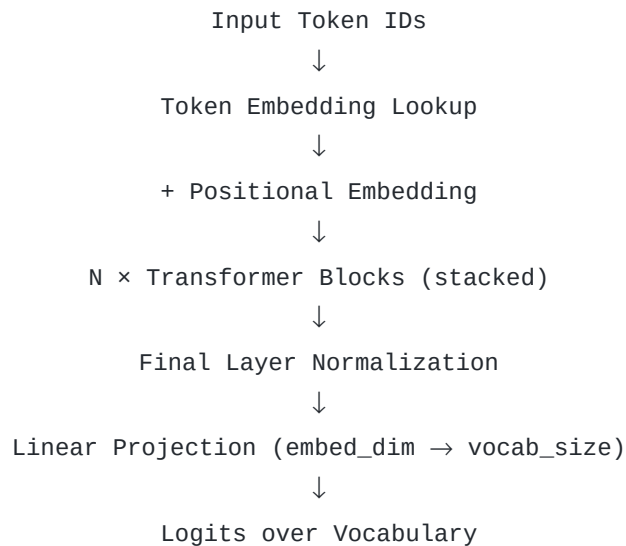
One Transformer block stacks all of the above in sequence:



5. GPT-Style Model Design

GPT (Generative Pre-trained Transformer) is a decoder-only Transformer. Unlike the encoder-decoder design of the original Transformer (used in translation), GPT uses only the decoder stack with causal masking, making it purely autoregressive — it generates one token at a time, always attending only to past context.

5.1 Architecture Overview



5.2 Key Hyperparameters

Hyperparameter	Description	GPT-2 Small Value
vocab_size	Number of unique tokens	50,257
context_length	Max tokens per sequence	1,024
embed_dim	Token embedding dimension	768
num_heads	Attention heads per block	12
num_layers	Stacked Transformer blocks	12
ffn_hidden_dim	FFN hidden size (4× embed)	3,072
dropout	Regularization rate	0.1

5.3 Output: Logits

The final linear layer projects each token's representation from `embed_dim` down to `vocab_size`, producing a vector of raw scores called logits. These logits represent the model's unnormalized confidence for each possible next token. Applying softmax converts logits to probabilities.

6. Pretraining

Pretraining is the initial training phase where the model learns general language patterns from a large text corpus using self-supervised learning. No human labels are required — the training signal comes from the data itself: predict the next token.

6.1 Objective: Next-Token Prediction

For every position in the input sequence, the model is trained to predict the token that comes next. Given tokens [t1, t2, t3, t4], the model predicts t2 from t1, t3 from [t1,t2], t4 from [t1,t2,t3], and so on — all in one forward pass thanks to causal masking.

6.2 Loss Function: Cross-Entropy

The model's prediction at each position is a probability distribution over the vocabulary. Cross-entropy loss measures how far this distribution is from the true one-hot distribution (where all probability mass sits on the correct next token). Minimizing this loss pushes the model to assign higher probability to correct tokens.

6.3 Optimizer: AdamW

AdamW is the standard optimizer for Transformer training. It combines Adam's adaptive per-parameter learning rates with decoupled weight decay regularization. Weight decay penalizes large weights, helping prevent overfitting. AdamW maintains first and second moment estimates for each parameter, which are saved as part of the checkpoint.

6.4 Perplexity

Perplexity (PPL) is the standard evaluation metric for language models. It is the exponent of the average cross-entropy loss: $PPL = \exp(\text{loss})$. A perplexity of N means the model is as confused as if it were uniformly choosing among N equally likely tokens at each step. Lower perplexity = better model. It provides an intuitive measure of how well the model has learned the language distribution.

6.5 Train / Validation Split

The corpus is split into a training set (used to update weights) and a validation set (never seen during training, used to monitor generalization). Tracking both losses reveals whether the model is underfitting (both losses high) or overfitting (train loss drops but val loss rises).

7. Text Generation Strategies

Once trained or weight-loaded, the model generates text autoregressively: it produces one token at a time, appends it to the context, and uses the updated context to predict the next token. Several strategies control the quality and diversity of generated text.

7.1 Greedy Decoding

The simplest strategy: always pick the token with the highest probability. This is fully deterministic but tends to produce repetitive, generic text because it always takes the 'safe' choice.

7.2 Temperature Scaling

Temperature is a scalar that modifies the logits before softmax. Dividing logits by a temperature T reshapes the probability distribution. Temperature < 1 sharpens the distribution (model is more confident, more conservative). Temperature > 1 flattens it (more uniform, more surprising outputs). Temperature $= 1$ leaves the distribution unchanged.

7.3 Top-k Sampling

Instead of sampling from the full vocabulary, top-k filtering restricts sampling to only the k highest-probability tokens. All other tokens are set to negative infinity before softmax. This prevents the model from sampling very low-probability garbage tokens while still allowing diversity among the top candidates.

7.4 Multinomial Sampling

After applying temperature and top-k filtering, the remaining logits are converted to probabilities via softmax. A token is then drawn by sampling from this multinomial distribution — higher-probability tokens are more likely to be picked, but lower-probability ones still have a chance. This produces varied, natural-sounding text.

7.5 EOS (End-of-Sequence) Handling

A special end-of-sequence token signals that generation should stop. When the model generates this token, the loop terminates. Without EOS handling, generation would continue indefinitely up to the context length limit.

8. GPT-2 Weight Loading

Rather than training from scratch (which requires massive compute), this project loads the publicly released pretrained weights from OpenAI's GPT-2 directly into the custom implementation. This validates that the architecture is correctly implemented and immediately gives access to a capable language model.

8.1 Available Variants

Model	Parameters	Layers	Heads	Embed Dim
GPT-2 Small	124M	12	12	768
GPT-2 Medium	355M	24	16	1,024
GPT-2 Large	774M	36	20	1,280
GPT-2 XL	1.5B	48	25	1,600

8.2 Weight Mapping

OpenAI's checkpoint uses different naming conventions and tensor layouts than the custom implementation. The loading utility maps each checkpoint key to the corresponding parameter in the custom model. Attention weight matrices in GPT-2 are stored as combined QKV projections and must be split before assignment. All tensor shapes are validated before copying to catch mismatches early.

8.3 What Gets Loaded

- Token embeddings (wte): vocabulary → embed_dim
- Positional embeddings (wpe): position → embed_dim
- All attention Q, K, V projection weights and biases
- All attention output projection weights
- All FFN layer weights (two linear layers per block)
- All LayerNorm gain (weight) and bias parameters
- Final output head (tied with token embeddings in GPT-2)

9. Checkpointing

Training large models is expensive in time and compute. Checkpointing saves the full training state to disk so training can be resumed from exactly where it left off if interrupted, or so the best model during training can be preserved.

What Is Saved

- Model state dict: all learned weight tensors
- Optimizer state dict: AdamW's first and second moment estimates for every parameter
- Current epoch and step count
- Training and validation loss history

Without saving the optimizer state, resumed training would behave as if starting fresh from the current weights with a cold optimizer — moment estimates would be reset to zero, causing an unstable initial phase. Saving the full optimizer state ensures seamless resumption.

10. SMS Spam Classification Fine-Tuning

Fine-tuning adapts a pretrained model to a specific downstream task with a much smaller labeled dataset. Here, the GPT model is fine-tuned for binary text classification: given an SMS message, predict whether it is spam (1) or ham — legitimate (0).

10.1 Why Fine-Tune Instead of Train From Scratch?

The pretrained GPT model already understands language — grammar, vocabulary, context. Fine-tuning leverages this knowledge and adapts it to the classification task in far fewer steps than training a classifier from random initialization would require.

10.2 Dataset: SMS Spam Collection

The UCI SMS Spam Collection dataset contains 5,574 English SMS messages labeled as spam or ham. It is downloaded and loaded into a Pandas DataFrame. Labels are converted: ham → 0, spam → 1.

10.3 Class Balancing

The dataset is imbalanced — ham messages significantly outnumber spam. Training on imbalanced data biases the model toward the majority class. To fix this, the majority class (ham) is randomly downsampled so both classes have equal representation. This ensures the model treats both classes equally during training.

10.4 Dataset Splits

The balanced dataset is shuffled and split into training, validation, and test sets. The training set updates the model weights. The validation set monitors for overfitting during training. The test set provides an unbiased final evaluation after training is complete.

10.5 SpamDataset

A custom PyTorch Dataset class wraps the SMS data. For each message it: tokenizes the text using the GPT tokenizer, truncates sequences longer than the maximum length, pads shorter sequences with the padding token to a fixed length, and returns the token ID tensor paired with the label.

10.6 Classification Head

GPT normally outputs logits over the full vocabulary at every position. For classification, only the representation at the last token position is used — this position has attended to the entire message. A linear layer maps this representation from `embed_dim` to 2 (binary: spam or ham). The model's loss is cross-entropy over these two classes.

10.7 Results

Epoch	Train Accuracy	Validation Accuracy
1	70.0%	72.5%
2	82.5%	85.0%

3	90.0%	90.0%
4	100.0%	97.5%
5	100.0%	97.5%

Training time: 15.82 minutes. The model achieved 97.5% validation accuracy, demonstrating that pretrained language representations transfer effectively to classification tasks even with modest fine-tuning data.

11. Instruction Fine-Tuning (SFT)

Instruction fine-tuning (also called Supervised Fine-Tuning, SFT) teaches the model to follow natural language instructions. This is what transforms a raw language model — which just predicts the next token — into an assistant-style model that responds helpfully to structured prompts.

11.1 What is SFT?

In SFT, the model is trained on a dataset of (instruction, response) pairs. The model is shown a formatted prompt containing an instruction (and optionally some input context) and learns to produce the correct response. The training signal is the same next-token prediction loss as pretraining, but now applied to curated instruction-response pairs rather than raw text.

11.2 Dataset

The dataset is `instruction-data.json` containing 1,100 entries from the `rasbt LLMs-from-scratch` repository. Each entry has three fields: `instruction` (what the model should do), `input` (optional additional context), and `output` (the ideal response).

11.3 Dataset Splits

Split	Proportion	Purpose
Training	85%	Update model weights
Test	10%	Final unbiased evaluation
Validation	5%	Monitor training, tune hyperparameters

11.4 Prompt Format

Each entry is formatted into a standardized prompt template before tokenization. This consistent structure teaches the model to recognize instruction boundaries and produce responses in the expected format:

```
Below is an instruction that describes a task. Write a response that appropriately
completes the request. ### Instruction: [instruction text] ### Input: [input text
or empty] ### Response: [output text]
```

11.5 InstructionDataset

A custom `Dataset` class pre-tokenizes each formatted entry end-to-end using `tiktoken` (the GPT-2 BPE tokenizer). The entire formatted string — prompt plus response — is tokenized as a single sequence. This is stored as a list of integer token IDs.

11.6 Custom Collate Function & Batching

Because different entries have different lengths, they cannot be naively stacked into a batch. A custom collate function handles this:

- Appends `<|endoftext|>` token as a sequence terminator to each entry.
- Pads all sequences in the batch to the length of the longest one using the pad token.
- Constructs input tensors as `tokens[:-1]` (all but last token).

- Constructs target tensors as `tokens[1:]` (all but first token — the next-token shift).
- Replaces padding token IDs in the target with `ignore_index=-100` so the loss function skips them entirely — the model is not penalized for predicting padding.
- Pre-configured via `functools.partial` with `device` and `allowed_max_length=1024` for clean reuse.

11.7 Why `ignore_index=-100`?

PyTorch's cross-entropy loss function has a built-in `ignore_index` parameter. When a target position contains this special value, that position's loss is set to zero and excluded from the mean. This is critical for variable-length batches: without it, the model would be penalized for its predictions at padding positions, which are meaningless filler and would corrupt the training signal.

12. Training & Evaluation Pipeline

After all data preparation is in place, the supervised training loop orchestrates the full fine-tuning process with careful tracking and reproducibility.

12.1 Supervised Training Loop

- Reproducibility seed set before all operations so results are deterministic.
- Model moved to the appropriate device (CPU or GPU) before training begins.
- Each epoch iterates over all batches from the training DataLoader.
- Optimizer gradients zeroed at the start of each batch step.
- Forward pass computes logits; loss computed against shifted targets.
- Backward pass computes gradients; optimizer updates all parameters.
- Periodic evaluation on the validation set tracks generalization.
- Tokens-seen counter accumulates total tokens processed for learning curve analysis.
- Wall-clock timing measured from start to finish for benchmarking.

12.2 Loss Visualization

Training and validation losses are plotted against both epochs and total tokens seen. Plotting against tokens (rather than just epochs) is more informative for comparing training runs with different batch sizes or evaluation frequencies, as it shows learning efficiency per unit of data processed.

12.3 Response Generation & Comparison

After fine-tuning, the model generates responses for every entry in the test set. Each generated response is stored alongside the reference output from the dataset. This produces a structured comparison that can be inspected qualitatively and passed to an automated scoring system.

12.4 Checkpoint Saving

The trained model's state dictionary is saved to disk with a sanitized filename derived from the model configuration. This allows the fine-tuned model to be reloaded for inference or further fine-tuning without repeating the training process.

13. Automated Response Evaluation with Ollama

Evaluating instruction-following quality at scale requires automated scoring. This project uses a locally running Ollama model as an evaluator: it reads each test entry (instruction + reference output + model response) and assigns an integer score.

13.1 What is Ollama?

Ollama is a tool for running open-source LLMs locally on your machine via a simple REST API. It is used here purely as an evaluation judge — it receives prompts describing the instruction and both the reference and generated response, then returns a quality score. This is known as the 'LLM-as-judge' evaluation paradigm.

13.2 Process Monitoring with psutil

Before any evaluation begins, the code checks whether the Ollama server process is actively running using the psutil library (Python System and Process Utilities). If Ollama is not running, an informative error is raised immediately rather than failing silently mid-evaluation. This guard ensures the evaluation environment is correctly set up before any API calls are made.

13.3 query_model — Local REST API

The query_model function sends HTTP POST requests to the local Ollama REST endpoint using Python's urllib.request (standard library, no external HTTP dependency). Key design decisions:

- temperature=0: fully deterministic responses — same prompt always yields same score.
- Fixed seed: ensures reproducibility across evaluation runs.
- Returns raw text: the model's full response string for downstream parsing.

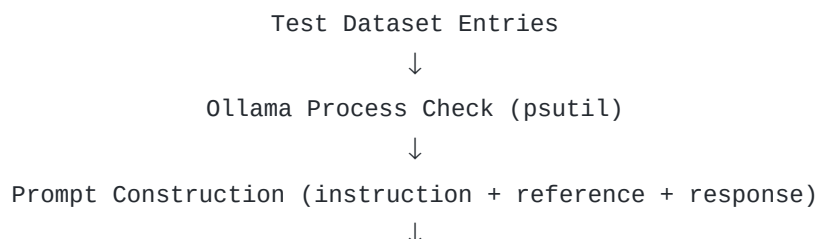
13.4 Integer Scoring Pipeline

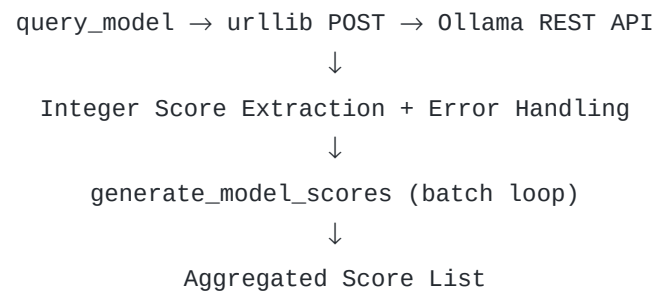
The evaluator LLM is prompted to respond with only an integer score (e.g., 1-100). The response text is parsed to extract this integer. Any conversion errors (if the model produces unexpected output) are caught and handled gracefully — the entry is skipped or assigned a default value rather than crashing the entire evaluation run.

13.5 Batch Scoring with generate_model_scores

generate_model_scores iterates over the full test dataset, calls query_model for each entry, parses the integer score, and accumulates results in a list. The final output is a list of numeric scores — one per test entry — which can be averaged or analyzed to summarize overall instruction-following quality.

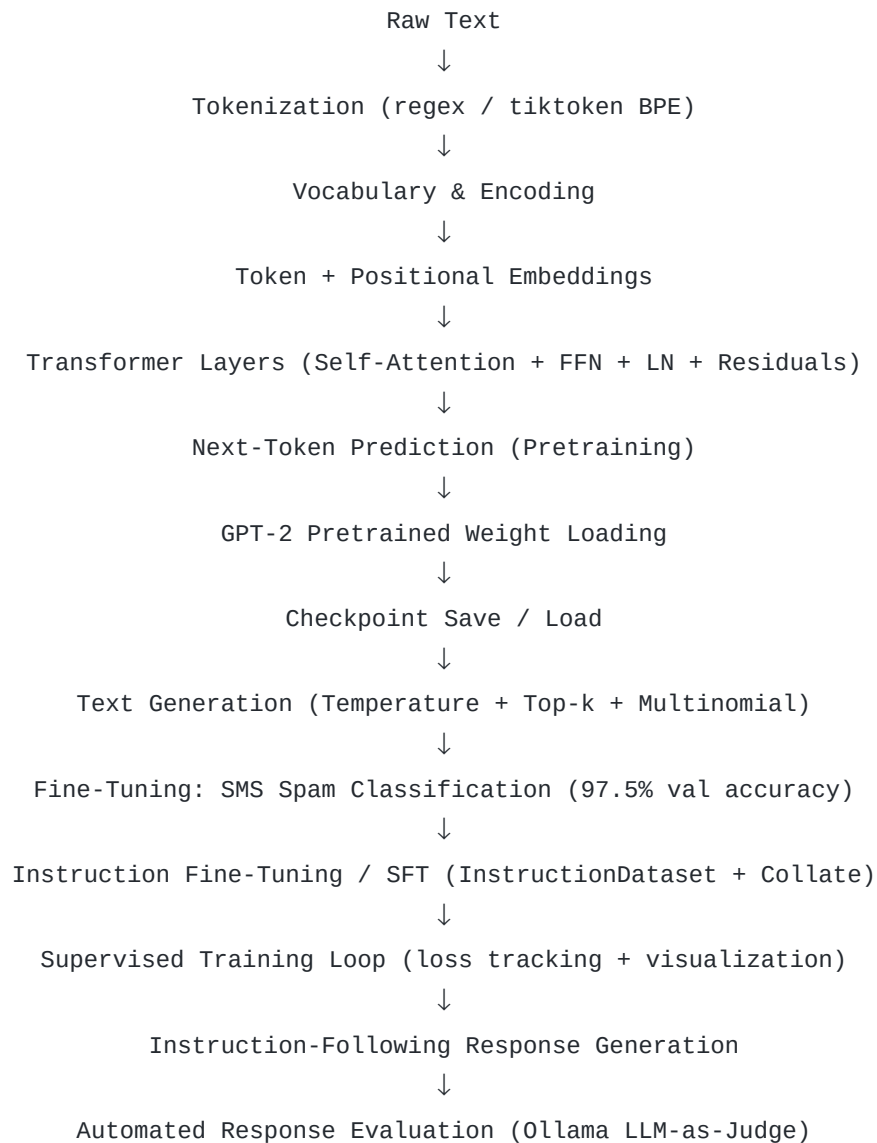
13.6 Evaluation Pipeline





14. End-to-End Pipeline Summary

The following summarizes every stage of the project in order, from raw text input to automated evaluation of instruction-following quality.



15. Key Concepts Glossary

Term	Definition
Token	The basic unit of text the model operates on. Can be a word, subword, or character.
Tokenizer	Converts raw text to integer token IDs and back.
Embedding	A dense vector representation of a token or position, learned during training.
Logits	Raw unnormalized scores output by the model before softmax.
Softmax	Converts logits to probabilities that sum to 1.
Cross-Entropy Loss	Measures how far the predicted distribution is from the true label.
Perplexity	$\exp(\text{loss})$ — intuitive measure of language model quality. Lower is better.
Self-Attention	Mechanism allowing each token to attend to all others in the sequence.
Causal Mask	Lower-triangular mask preventing tokens from attending to future positions.
Multi-Head Attention	Multiple parallel attention computations capturing diverse relationships.
Query / Key / Value	Projections used in attention: Q asks questions, K provides keys, V provides content.
Residual Connection	Skip connection adding input directly to sub-layer output, easing gradient flow.
Layer Normalization	Normalizes activations per token across feature dimension for stable training.
GELU	Smooth activation function used inside the feed-forward network.
AdamW	Adam optimizer with decoupled weight decay. Standard for Transformer training.
Fine-Tuning	Adapting a pretrained model to a specific task using a small labeled dataset.
SFT	Supervised Fine-Tuning — training on (instruction, response) pairs.
<code>ignore_index=-100</code>	Tells PyTorch loss to skip positions with this target value (used for padding).
Collate Function	Custom batching function that pads variable-length sequences in a DataLoader.
Temperature	Scalar controlling output randomness: <1 conservative, >1 diverse.
Top-k Sampling	Restricts token sampling to the k highest-probability candidates.
Ollama	Tool for running open-source LLMs locally, used here as an evaluation judge.
LLM-as-Judge	Using a language model to score another model's outputs automatically.
Checkpoint	Saved state of model weights and optimizer, enabling training resumption.
psutil	Python library for process and system monitoring, used to verify Ollama is running.
BPE	Byte-Pair Encoding — subword tokenization algorithm used by tiktoken/GPT-2.
Transfer Learning	Applying knowledge from pretraining on large data to a smaller downstream task.

Decoder-Only	Transformer variant using only the decoder stack with causal masking (GPT style).
--------------	---

16. About the Author

■■■ Arjun

Second-Year B.E. Information Technology Student

This project was designed, architected, and built entirely from scratch by Arjun as a foundational deep learning portfolio project. Every component — from the regex tokenizer and custom attention mechanisms to the GPT-2 weight loading pipeline, instruction fine-tuning collate functions, and Ollama-based automated evaluation — was implemented independently in PyTorch, without relying on high-level frameworks like HuggingFace Transformers.

The goal was to move beyond API calls and surface-level usage to develop a deep, first-principles understanding of how modern LLMs are built, trained, and evaluated.

This documentation was written to accompany the public GitHub repository so that other learners and developers can understand the theory and design decisions behind each component without getting lost in implementation details. The source code provides the 'how'; this document provides the 'why'.

Project Scope

- bullet Full GPT-style Transformer implemented from first principles
 - bullet GPT-2 weight loading across all four official model sizes
 - bullet Complete pretraining pipeline with loss tracking and visualization
 - bullet SMS spam classification fine-tuning (97.5% validation accuracy)
 - bullet Instruction fine-tuning with custom Dataset and collate function
 - bullet Automated evaluation using a local Ollama LLM-as-judge pipeline
-

Built from scratch. Every line written with intent.

— *Arjun*